

```
Test01.java x TestRunnerClass.java TestAssertion.java SingletonDP.java Singleton.java v ...
1 package PracticeSet.JUNITtesting;
2
3
4 import org.junit.jupiter.api.*;
5 import org.junit.jupiter.params.ParameterizedTest;
6 import org.junit.jupiter.params.provider.ValueSource;
7
8
9 import java.util.Collection;
10 import java.util.stream.Stream;
11
12 import static org.junit.jupiter.api.Assertions.assertEquals;
13 import static org.junit.jupiter.api.Assertions.assertTrue;
14 import static org.junit.jupiter.api.DynamicTest.dynamicTest;
15
16
17 public class Test01 {
18     @Test
19     public void Testcase1() {
20         String str = "Himanshu";
21         assertEquals(expected: "Himanshu", str);
22     }
23     private int testcase2() { 1 usage
24         return 10;
25     }
26     @Test
27     void Testcase2() {
```

```
Test01.java x TestRunnerClass.java TestAssertion.java SingletonDP.java Singleton.java v ...
17 public class Test01 {
26     @Test
27     void Testcase2() {
28         boolean condition = "true".equalsIgnoreCase(System.getProperty("runTest"));
29         Assumptions.assumeTrue(condition, message: "as the condition is not met skip test case");
30         int result = testcase2();
31         Assertions.assertEquals(expected: 10, result, message: "value need to be 10");
32     }
33
34
35     @ParameterizedTest
36     @ValueSource(ints = {100, 25, 30, 70, 40})
37     void testSquare(int value) {
38         int result = square(value);
39         assertEquals(expected: value * value, result, message: " if wrong");
40     }
41
42
43     private int square(int number) { 1 usage
44         return number * number;
45     }
46
47
48     @TestFactory
49     Collection<DynamicTest> DynamicTestCase() {
50         return TestCases().map(val ->
51             dynamicTest(displayName: "Dynamic Test: " + val, () -> assertTrue(condition: val % 2 ==
52             ));
53     }
54 }
```

```
Test01.java x TestRunnerClass.java TestAssertion.java SingletonDP.java Singleton.java v ...
17 public class Test01 {
48     @TestFactory
49     Collection<DynamicTest> dynamicTestCase() {
50         return TestCases().map(val ->
51             dynamicTest( displayName: "Dynamic Test: " + val, () -> assertTrue( condition: val % 2 ==
52             ).toList());
53     }
54     @Test
55     @Tag("slow")
56     void slowTest() {
57         assertTrue( condition: true, message: "slow test method");
58     }
59
60
61     @Test
62     @Tag("fast")
63     void fastTest() {
64         assertTrue( condition: true, message: "fast test method");
65     }
66
67
68     @Test
69     @Tag("fast")
70     @Tag("integration")
71     void fastIntegrationTest() {
72         assertTrue( condition: true, message: "fast integration test methid");
73     }
}
```

```
Test01.java x TestRunnerClass.java TestAssertion.java SingletonDP.java Singleton.java v ...
17 public class Test01 {
75
76     @Test
77     void test2() {
78         System.out.println("sample test method");
79     }
80
81 }
82
83     @BeforeAll
84     static void setupBeforeAll() {
85         System.out.println("run it b4 all tests");
86     }
87
88
89     @AfterAll
90     static void cleanupAfterAll() {
91         System.out.println("run it after all tests");
92     }
93
94
95     @BeforeEach
96     void setupBeforeEach() {
97         System.out.println("run it b4 each test");
98         int val1 = 10;
99         int val2 = 5;
100 }
```

```
Test01.java x TestRunnerClass.java TestAssertion.java SingletonDP.java Singleton.java
17 public class Test01 {
105     @AfterEach
106     void cleanupAfterEach() {
107         System.out.println("run it after each test");
108     }
109
110
111
112     @Test
113     void test() {
114         int val1 = 30;
115         int val2 = -25;
116         System.out.println("product of two nos");
117         int result = val1 + val2;
118         assertEquals(expected: 5, result, message: "actual product should be 50");
119     }
120
121     @
122     private Stream<Integer> TestCases() { 1 usage
123         return Stream.of(...values: 100, 200, 300, 400, 5000);
124     }
125
126 }
127
128
129
130
```

```
IntroductionToSpringBoot main TestCase10.hamcrestMethod
Project test
  java
    com.amazon.himanshu.wee
    PracticeSet.JUNITtesting
      AllTestSuite
      CalculatorTest
      Test01
      Test09
      TestAssertion
      TestCase02
      TestCase10
      TestCase11.java
      TestCase12
      TestCase13
      TestRunnerClass
  target
    .gitignore
    FileName00.txt
    FileName01.txt
    FileName02.txt
    FileName03.txt
    FileName04.txt
    FileName05.txt
    HELP.md
    introductionToSpringBoot.iml
TestCase02.java x TestCase10.java TestCase11.java TestCase12.java TestCase13.java
7 class TestCase02 {
8
9
10
11     @Test
12     @Tag("firstPriority")
13     void testMethod01() {
14         assertEquals(expected: 2, actual: 1 + 1);
15     }
16
17     @Test
18     @Tag("firstPriority")
19     void runTestcase02() {
20         assertEquals(expected: "hello", actual: "he" + "llo");
21     }
22
23     @Test
24     @Tag("fastTag")
25     void testMethod03() {
26         assertEquals(expected: true, actual: 5 > 3);
27     }
28
29     @Test
30     @Tag("slowTag")
31     void runTestcase04() {
32         assertEquals(expected: 9, actual: 3 * 3);
33     }
34 }
```

```
1 package PracticeSet.JUNITtesting;
2
3
4 import org.junit.jupiter.api.Test;
5 import static org.hamcrest.MatcherAssert.assertThat;
6 import static org.hamcrest.Matchers.*;
7
8
9 public class TestCase10 {
10     @Test
11     public void hamcrestMethod() {
12         String str1 = "Apple";
13         String str2 = "Apple";
14         assertThat(str1, equalToIgnoringCase(str2));
15     }
16 }
17
```

```
5
6 import org.junit.jupiter.api.Test;
7 import static org.hamcrest.MatcherAssert.assertThat;
8 import static org.hamcrest.Matchers.hasToString;
9
10 public class TestCase11 {
11     @Test
12     public void method1() {
13         Customer customer = new Customer(fName: "John", lName: "Abraham");
14         String str = customer.toString();
15         assertThat(customer, hasToString(str));
16     }
17 }
18
19 class Customer {
20     private String fName;
21     private String lName;
22     public Customer(String fName, String lName) {
23         this.fName = fName;
24         this.lName = lName;
25     }
26     public String getfName() {
27         return fName;
28     }
29     public String getlName() {
30         return lName;
31     }
32 }

```

Project ▾

- test
 - java
 - com.amazon.himanshu.wee
 - PracticeSet.JUNITtesting
 - AllTestSuit
 - CalculatorTest
 - Test01
 - Test09
 - TestAssertion
 - TestCase02
 - TestCase10
 - TestCase11.java
 - Customer
 - TestCase11
 - TestCase12**
 - TestCase13
 - TestRunnerClass
- target
- .gitignore
- FileName00.txt
- FileName01.txt
- FileName02.txt
- FileName03.txt
- FileName04.txt
- FileName05.txt

TestCases02.java TestCases10.java × TestCases11.java **TestCases12.java ×** TestCases13.java ▾

```
1 package PracticeSet.JUNITtesting;
2
3
4 import org.junit.jupiter.api.Test; //collection Matcher
5 import java.util.ArrayList;
6 import java.util.List;
7 import static org.hamcrest.MatcherAssert.assertThat;
8 import static org.hamcrest.Matchers.empty;
9
10
11 public class TestCase12 {
12     @Test
13     public void method1() {
14         List<String> custList = new ArrayList<>();
15         assertThat(custList, empty());
16     }
17 }
18
```

Project ▾

- test
 - java
 - com.amazon.himanshu.wee
 - PracticeSet.JUNITtesting
 - AllTestSuit
 - CalculatorTest
 - Test01
 - Test09
 - TestAssertion
 - TestCase02
 - TestCase10
 - TestCase11.java
 - Customer
 - TestCase11
 - TestCase12
 - TestCase13**
 - TestRunnerClass
- target
- .gitignore
- FileName00.txt
- FileName01.txt
- FileName02.txt
- FileName03.txt
- FileName04.txt
- FileName05.txt

TestCases02.java TestCases10.java TestCases11.java TestCases12.java **TestCases13.java ×** ▾

```
1 package PracticeSet.JUNITtesting;
2
3
4 import org.junit.jupiter.api.Test;
5 import java.util.Arrays;
6 import java.util.List;
7 import static org.hamcrest.MatcherAssert.assertThat;
8 import static org.hamcrest.Matchers.hasSize;
9
10
11 public class TestCase13 {
12     @Test
13     public void method2() {
14         List<String> custList = Arrays.asList("john", "Mary", "Sheik", "Singh");
15         assertThat(custList, hasSize(4));
16     }
17 }
18
```